Criptografia Homomórfica CKKS para Aprendizado Federado Preservador de Privacidade

Guia conceitual para praticantes de FL

Yuri Dimitre Dias de Faria
Instituto de Computação – UNICAMP
Orientador: Prof. Dr. Allan Mariano de Souza

Junho de 2026

1 O problema: privacidade na agregação federada

No Aprendizado Federado (FL), um servidor central coordena o treinamento de um modelo global sem coletar os dados brutos dos clientes. Cada cliente k computa um gradiente g_k sobre seus dados locais e o transmite para o servidor, que agrega as atualizações e devolve o novo modelo. O pressuposto de privacidade é que apenas os gradientes, e não os dados de treinamento, saem dos dispositivos.

Esse pressuposto é fraco. Ataques de inversão de gradiente [4] reconstroem imagens de entrada com alta fidelidade a partir de g_k. Ataques de inferência de pertença [5] determinam se um exemplo específico participou do treinamento. Em aplicações médicas, financeiras ou governamentais, onde a lei exige proteção dos dados individuais, expor os gradientes ao servidor é juridicamente problemático mesmo quando o servidor é considerado honesto.

A Criptografia Homomórfica (HE) elimina esse vetor de ataque: os clientes enviam *gradientes cifrados*, o servidor agrega matematicamente sem acessar os valores, e apenas o detentor da chave secreta decifra o resultado final. A propriedade fundamental que torna isso possível é o **homomorfismo**: a soma dos textos cifrados, ao ser decifrada, produz a soma dos textos claros correspondentes, sem que o servidor precise de qualquer chave.

2 Criptografia Homomórfica: o essencial em uma página

Texto claro, texto cifrado e par de chaves. Um esquema de criptografia homomórfica define três objetos fundamentais. O **texto claro** (plaintext) é o dado original, por exemplo o vetor de gradientes $g_k \in \mathbb{R}^d$. O **texto cifrado** (ciphertext) é uma representação cifrada de g_k, legível apenas por quem detém a chave secreta. O **par de chaves** consiste em uma chave pública pk, que qualquer cliente usa para cifrar, e uma chave secreta sk, que apenas a entidade de confiança mantém para decifrar. A assimetria é a garantia de privacidade: clientes cifram com pk, o servidor manipula textos cifrados, e apenas o detentor de sk vê os valores.

O homomorfismo aditivo. A propriedade central para FL é a **adição homomórfica**: dado um operador de adição $\oplus$ sobre textos cifrados,

$$\mathrm{Dec}(\mathrm{Enc}(g_1) \oplus \mathrm{Enc}(g_2)) \approx g_1 + g_2, \tag{1}$$

onde Enc e Dec são as operações de cifragem e decifragem, respectivamente. A igualdade em (1) é aproximada: todo esquema de HE prático incorpora um pequeno ruído criptográfico cujos efeitos são discutidos na seção 4.

O esquema CKKS. O CKKS [1], também denominado HEAAN na literatura, é o esquema de HE mais adequado para FL porque opera nativamente sobre **números reais e complexos aproximados**. Outros esquemas (BFV, BGV) operam sobre inteiros exatos, o que obriga a quantização prévia dos gradientes com perda de precisão adicional. O CKKS codifica os gradientes diretamente com um fator de escala Δ e aceita um pequeno erro de aproximação como custo intrínseco, similar ao erro de ponto flutuante de 64 bits.

Empacotamento de múltiplos valores por criptograma. O CKKS não cifra um único valor por vez. Cada criptograma acomoda simultaneamente $N/2$ valores independentes, denominados **slots**, onde N é o grau do polinômio anel usado internamente. Para $N = 4096$, um criptograma carrega 2048 gradientes; para $N = 8192$, carrega 4096. Isso reduz o número de operações de rede e de cifragem na proporção $N/2$ em relação a cifrar escalar a escalar.

Por que só adição é suficiente para FL. O algoritmo FedAvg, e mais geralmente qualquer forma de agregação fedrerada por soma ponderada, calcula

$$G = \sum_{k=1}^{K} w_k \, g_k,$$

onde os pesos $w_k = n_k / \sum_j n_j$ são escalares conhecidos pelo servidor. Essa operação é inteiramente aditiva do ponto de vista criptográfico: basta que o cliente k pré-multiplique g_k por w_k *antes* de cifrar, e o servidor some os criptogramas. Nenhuma multiplicação entre dois textos cifrados é necessária. Isso é relevante porque multiplicação homomórfica entre dois textos cifrados é dispendiosa: requer relinearização e reescalonamento (*rescaling*), que aumentam o custo de processamento e consomem o orçamento de ruído do esquema. A restrição ao regime somente-aditivo elimina completamente essas operações.

3 O fluxo do FL cifrado

Preparação da rodada. Uma entidade de confiança (que pode ser o próprio servidor sob determinadas premissas de segurança, ou uma terceira parte, ou um comitê de clientes) gera o par de chaves $(\mathrm{pk}, \mathrm{sk})$. A chave pública pk é distribuída a todos os participantes; a chave secreta sk permanece protegida.

Cifragem no cliente. Cada cliente k computa o gradiente local $g_k \in \mathbb{R}^d$, divide-o em segmentos de tamanho $N/2$, codifica cada segmento como um polinômio com escala Δ, e cifra com pk. O resultado é uma lista de textos cifrados $\{\mathrm{ct}_k^{(i)}\}$. O cliente envia apenas esses criptogramas ao servidor; g_k nunca sai em texto claro.

Agregação no servidor. O servidor recebe $\{\mathrm{ct}_k^{(i)}\}$ de todos os K clientes e soma os criptogramas correspondentes ao mesmo segmento i:

$$\mathrm{ct}_{\mathrm{agg}}^{(i)} = \bigoplus_{k=1}^{K} \mathrm{ct}_k^{(i)}. \tag{2}$$

A operação $\oplus$ em (2) é a adição coeficiente a coeficiente dos polinômios que compõem cada criptograma, que não requer sk. A fig. 1 ilustra o fluxo completo.

Decifragem e atualização. O servidor envia $\{\mathrm{ct}_{\mathrm{agg}}^{(i)}\}$ ao detentor de sk, que decifra, decodifica e recupera uma aproximação de $\sum_k g_k$. Essa agregação é usada para atualizar o modelo global, que é distribuído em texto claro para a próxima rodada.

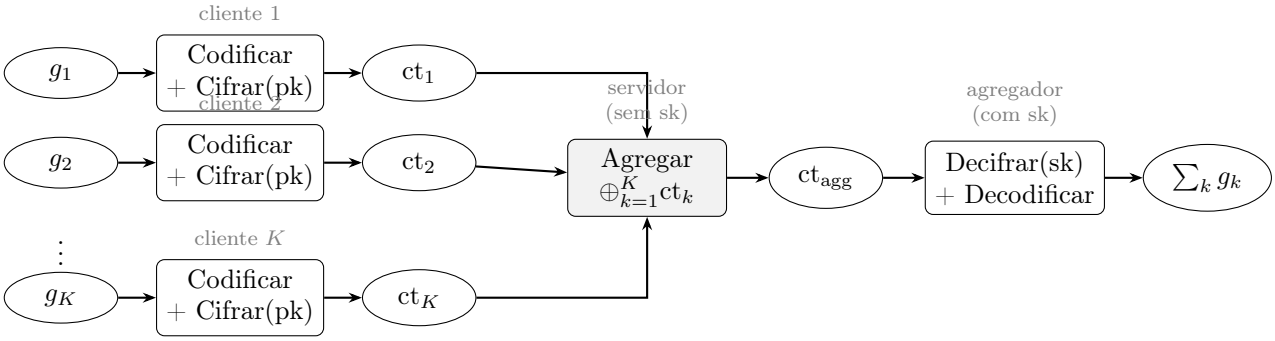


Figura 1: Fluxo do FL com cifragem homomórfica. O servidor agrega criptogramas sem acesso à chave secreta; o gradiente em texto claro nunca trafega pela rede.

4 O que custa: precisão, ruído e comunicação

Escala Δ e erro de codificação. Antes de cifrar, cada valor real $g \in \mathbb{R}$ é multiplicado por Δ e arredondado para um inteiro. O erro introduzido por esse arredondamento é delimitado por

$$|\varepsilon_{\mathrm{cod}}| \leq \frac{1}{2\Delta}. \tag{3}$$

Aumentar Δ reduz o erro de codificação, mas exige um módulo aritmético q maior para que os coeficientes do polinômio permaneçam dentro do intervalo válido. Valores típicos são $\Delta = 2^{40}$ a 2^{50}, garantindo de 12 a 15 dígitos decimais de precisão antes da agregação.

Ruído criptográfico e crescimento com K. Todo esquema RLWE incorpora um ruído e nos coeficientes do criptograma durante a cifragem. Esse ruído é intencional: ele é a razão pela qual o adversário não consegue recuperar a chave secreta. Na decifragem, o ruído criptográfico se soma ao erro de codificação conforme a equação de erro total por slot:

$$\hat{g} = g + \varepsilon_{\mathrm{cod}} + \frac{e}{\Delta}, \tag{4}$$

onde e é amostrado de uma distribuição gaussiana discreta com desvio padrão σ_e.

Ao agregar K clientes, os ruídos individuais $e^{(k)}$ são independentes e de média zero. A variância total é a soma das variâncias; portanto, o desvio padrão do erro agregado por slot cresce como

$$\sigma_{\mathrm{agg}} \approx \frac{\sqrt{K}\,\sigma_e}{\Delta}. \tag{5}$$

O crescimento sublinear de $\sqrt{K}$ é uma vantagem em relação a mecanismos de ruído diferencial, que adicionam ruído proporcional a K. Ainda assim, em federações com centenas de clientes, o termo $\sqrt{K}\,\sigma_e/\Delta$ deve permanecer muito abaixo do sinal útil, o que impõe um limite inferior sobre Δ.

Grau N: capacidade, segurança e custo de banda. O parâmetro N regula três aspectos do sistema de forma simultânea. Primeiro, o número de slots por criptograma é $N/2$: para um gradiente de dimensão d, o cliente transmite $\lceil d/(N/2) \rceil$ criptogramas. Segundo, a resistência a ataques contra o problema RLWE subjacente aumenta com N para um módulo

q fixo; valores comumente recomendados são $N \in \{4096, 8192, 16384\}$ para níveis de segurança de 128, 192 e 256 bits, respectivamente. Terceiro, o tamanho de cada criptograma é $2 \times (12 + 8N)$ bytes, o que para $N = 8192$ resulta em aproximadamente 131 KB por criptograma; a expansão de tamanho é o principal custo de comunicação da abordagem.

A tabela 1 resume os efeitos de variar cada parâmetro.

Tabela 1: Efeitos de aumentar cada parâmetro do CKKS no contexto de FL.

Parâmetro	**Precisão**	**Segurança**	**Custo de banda**
Δ maior	melhora (↑)	neutro	neutro (precisa q maior)
N maior	mais slots	melhora (↑)	aumenta (↑)
K maior	piora ($\sqrt{K}$)	neutro	aumenta (↑)

Comparação com alternativas. A Privacidade Diferencial (DP) oferece proteção formal a custo computacional baixo, mas injeta ruído que escala linearmente com K e não oculta o valor exato do gradiente do servidor. A Computação Segura Multipartidária (MPC) protege contra servidores desonestos sem expansão de mensagem, mas exige múltiplas rodadas de comunicação. O HE com CKKS é preferível quando o servidor deve ser incapaz de observar qualquer gradiente individual, a latência de múltiplas rodadas é proibitiva, e a precisão em ponto flutuante importa.

5 Fundação de segurança e a biblioteca implementada

O problema RLWE. A segurança do CKKS repousa sobre a hipótese do **Aprendizado com Erros em Anéis** (RLWE) [2]. Dado um polinômio a sorteado uniformemente e um polinômio de ruído e com coeficientes gaussianos pequenos, o RLWE afirma que o par $(a, a \cdot s + e)$, onde s é a chave secreta, é computacionalmente indistinguível de dois polinômios uniformes independentes. Em outras palavras: mesmo observando o criptograma (c_0, c_1) inteiro, um adversário sem sk não consegue recuperar g_k em tempo polinomial, assumindo que RLWE é intratável. A escolha dos parâmetros (N, q) é guiada por estimativas de custo de ataques de retículo (LWE Estimator), e valores como $N = 8192$ com $q < 2^{62}$ atingem 128 bits de segurança.

A biblioteca `ckks_core`. O *backend* C++ `ckks_core`, implementado para suportar os experimentos da dissertação, opera em regime somente-aditivo com um único primo q em vez de uma cadeia RNS completa. Essa escolha simplifica a implementação e reduz o *overhead* de cifragem ao mínimo necessário para a agregação federada, onde nenhuma multiplicação entre textos cifrados é realizada. As multiplicações polinomiais internas (cifragem e decifragem) usam a Transformada Numérica Teórica (NTT) com complexidade $O(N \log N)$, e toda a aritmética modular é implementada com redução de Montgomery para eliminar divisões do caminho crítico. Os detalhes de implementação de baixo nível estão documentados no relatório técnico complementar. A interface Python (`ckks_python`) e a camada de alto nível para Flower (`ckks_federated`) expõem esses mecanismos de forma transparente ao praticante de FL.

Síntese. O CKKS homomórfico aditivo é uma solução técnica concreta para o problema de privacidade na agregação federada. Seus três parâmetros essenciais, N, Δ e K, devem ser calibrados em conjunto: N determina a capacidade de empacotamento e o custo de banda; Δ controla a precisão; K amplifica o ruído por $\sqrt{K}$. Para as dimensões típicas de modelos de FL (dezenas de milhares a poucos milhões de parâmetros) e federações de até algumas centenas

de clientes, os parâmetros $N = 8192$ e $\Delta = 2^{40}$ a 2^{50} produzem ruído agregado abaixo de 10^{-4} por slot, compatível com a precisão de treinamento de modelos de aprendizado de máquina.

Referências

[1] J. H. Cheon, A. Kim, M. Kim, e Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," em *ASIACRYPT 2017*, LNCS vol. 10624, Springer, pp. 409–437.

[2] V. Lyubashevsky, C. Peikert, e O. Regev, "On ideal lattices and learning with errors over rings," em *EUROCRYPT 2010*, LNCS vol. 6110, Springer, pp. 1–23.

[3] Microsoft Research, *SEAL: Simple Encrypted Arithmetic Library – Manual*, v. 4.1, 2023. https://github.com/microsoft/SEAL

[4] J. Geiping, H. Bauermeister, H. Dröge, e M. Moeller, "Inverting gradients – How easy is it to break privacy in federated learning?" em *NeurIPS 2020*.

[5] R. Shokri, M. Stronati, C. Song, e V. Shmatikov, "Membership inference attacks against machine learning models," em *IEEE S&P 2017*.